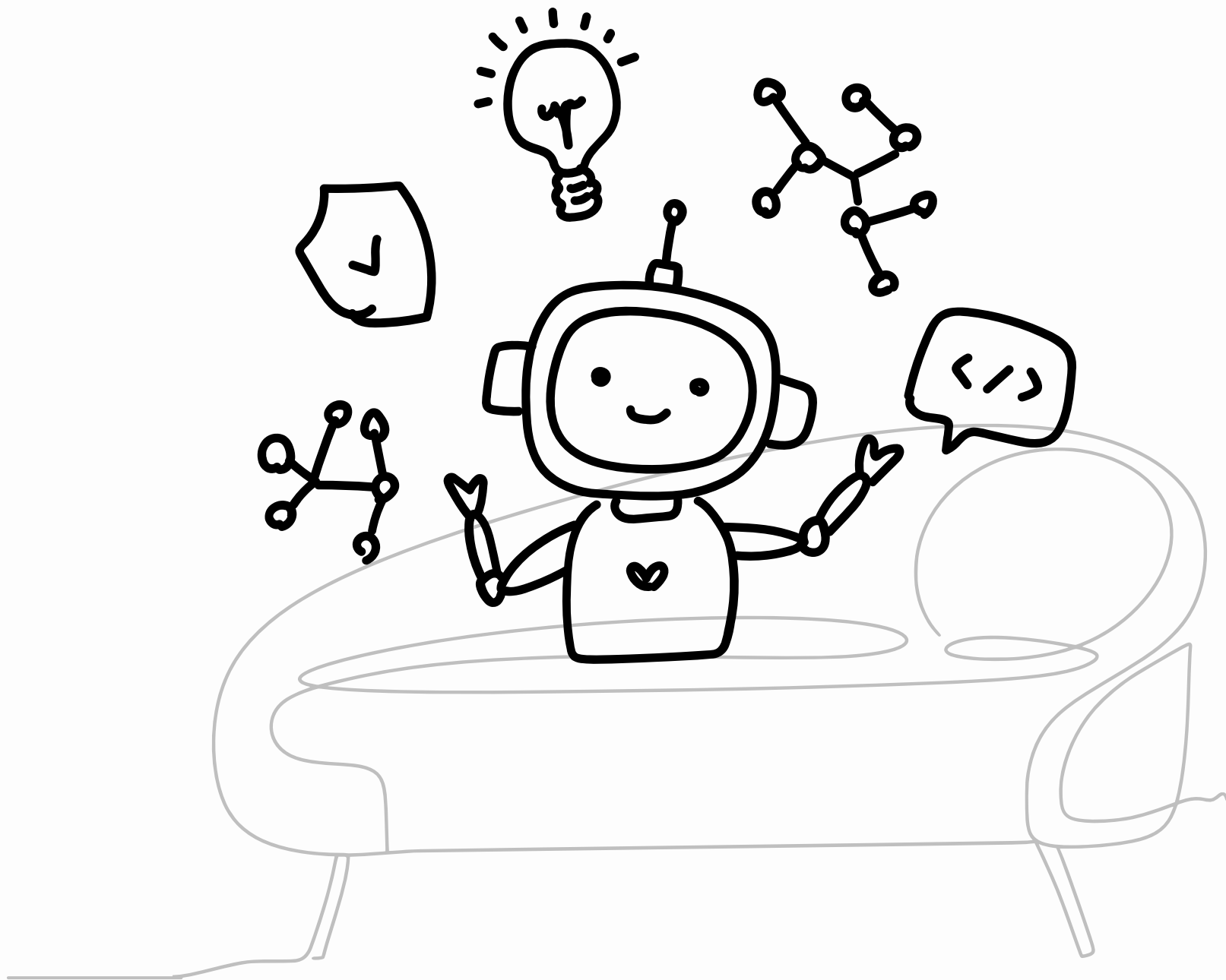


Intro GenAI *for the Humanities*

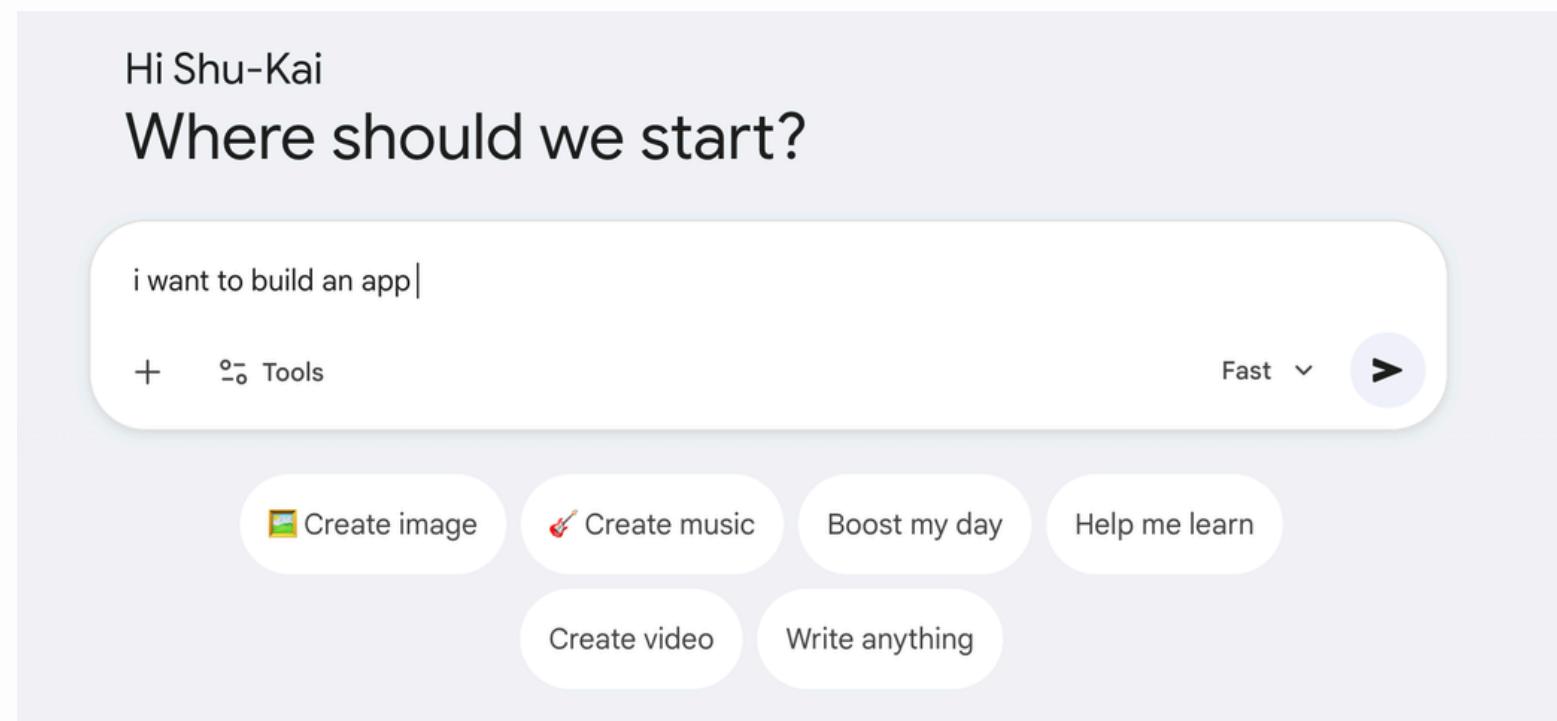
第八週

謝舒凱



提醒大家

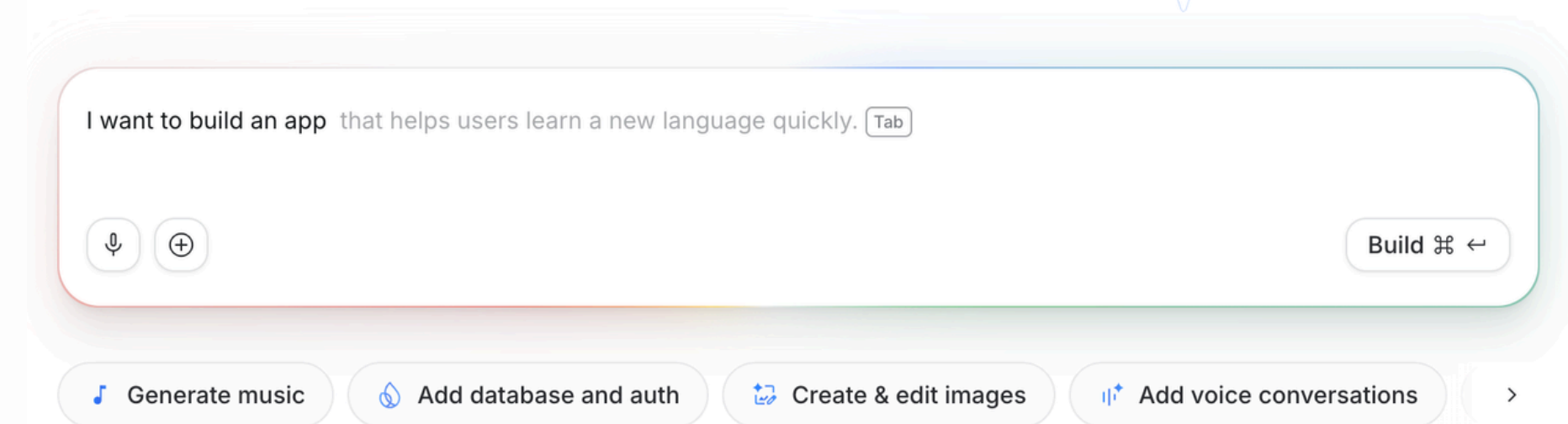
不思考是一個誘人的陷阱（就像躺在沙發上看電視）



<https://gemini.google.com/app>

tab! tab! tab!

Build your ideas with Gemini



<https://aistudio.google.com/apps>

提醒大家

學會怎麼學，比起學什麼還重要 *meta-learning vs. learning*



coursera Explore ▾ My Learning Degrees learning how to learn

🌟 AI Overview

Understanding how to learn effectively

To master the skill of learning how to learn, focus on **developing mental tools and strategies** that improve retention and comprehension. Start with beginner-friendly courses that cover **learning strategies, time management, and lifelong learning**. Consider your goals: whether you want to enhance personal development, improve productivity, or teach others.

[Show more](#)

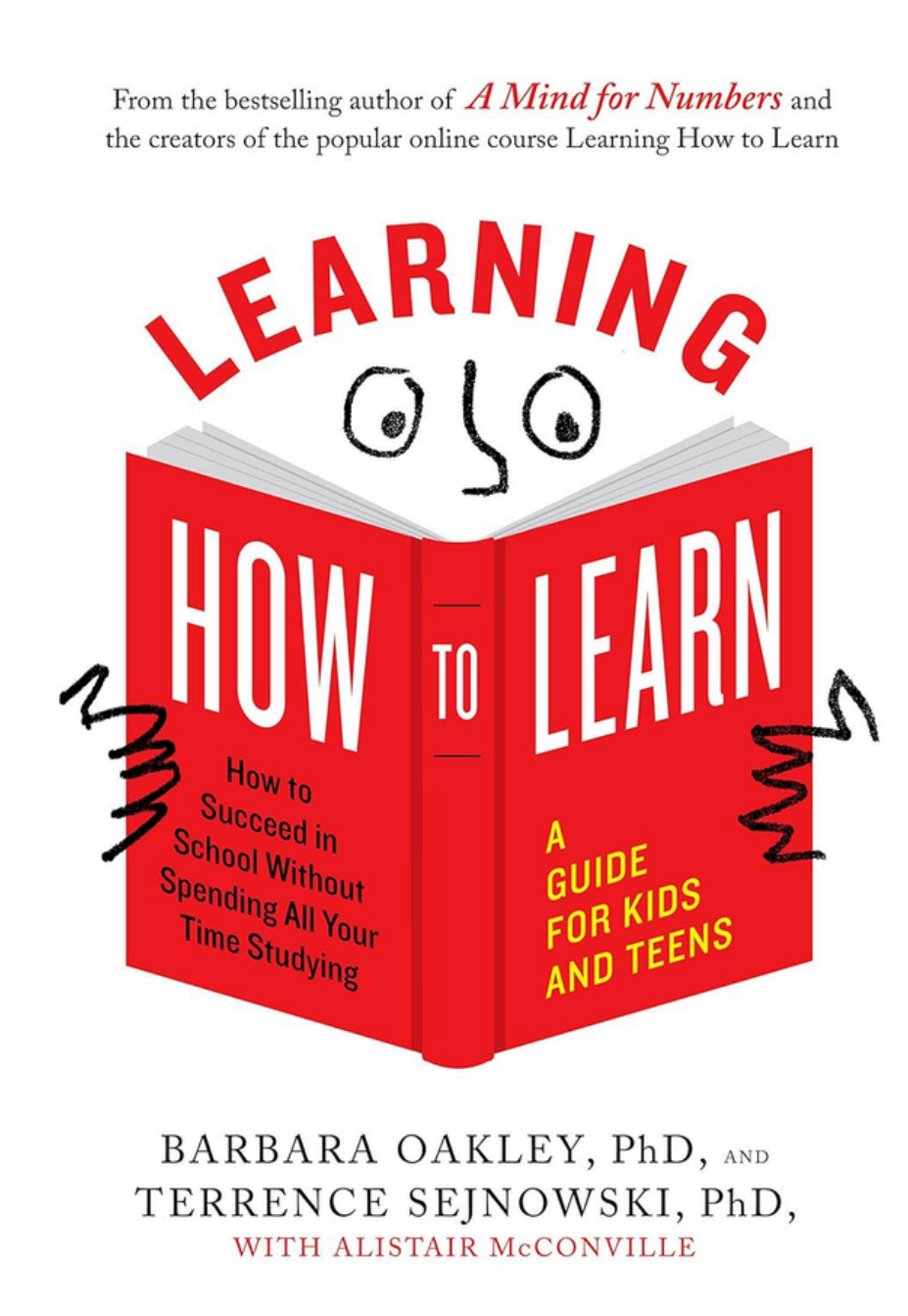
Top courses to get started:

Deep Teaching Solutions
Learning How to Learn: Powerful mental tools...
Best for: beginners, learners with one to four weeks availability, and those seeking personal development skills to master tough subjects

Arizona State University
Learning How To Learn for Youth
Best for: youth learners, beginners, and those interested in productivity and human learning strategies

Deep Teaching Solutions
学会如何学习：帮助你掌握复杂学科的强大...
Best for: mixed-level learners, those with one to three months availability, and learners focused on collaboration and lifelong learning

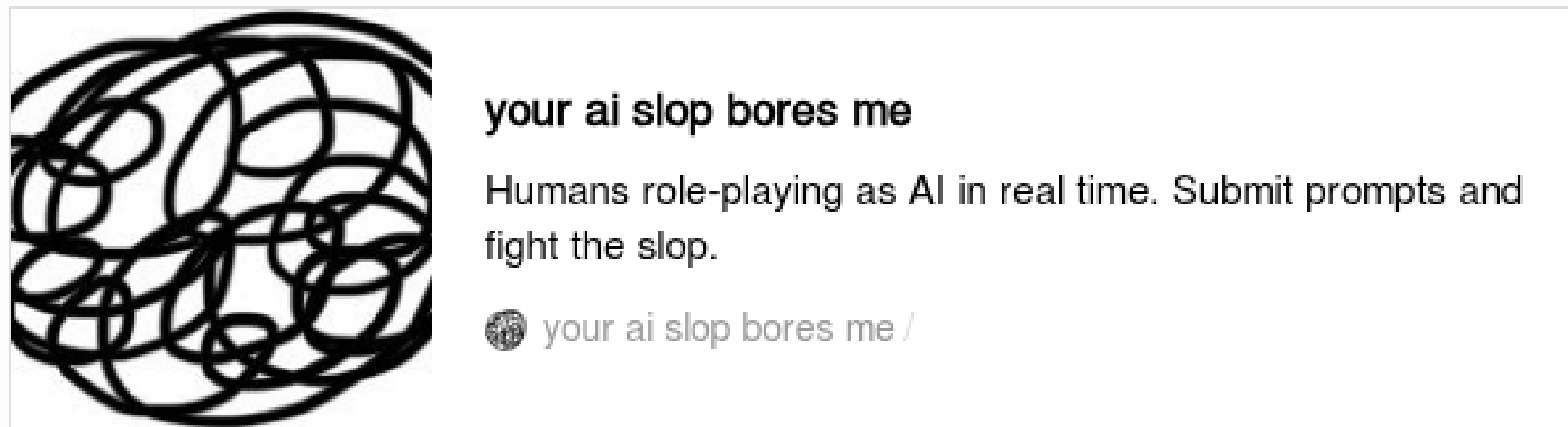
UNSW Sydney (The Uni...
Learning to Teach Online
Best for: beginners, learners with one to three months availability, and aspiring online educators looking to teach digitally



為 9-12 歲

多觀察與理解人性

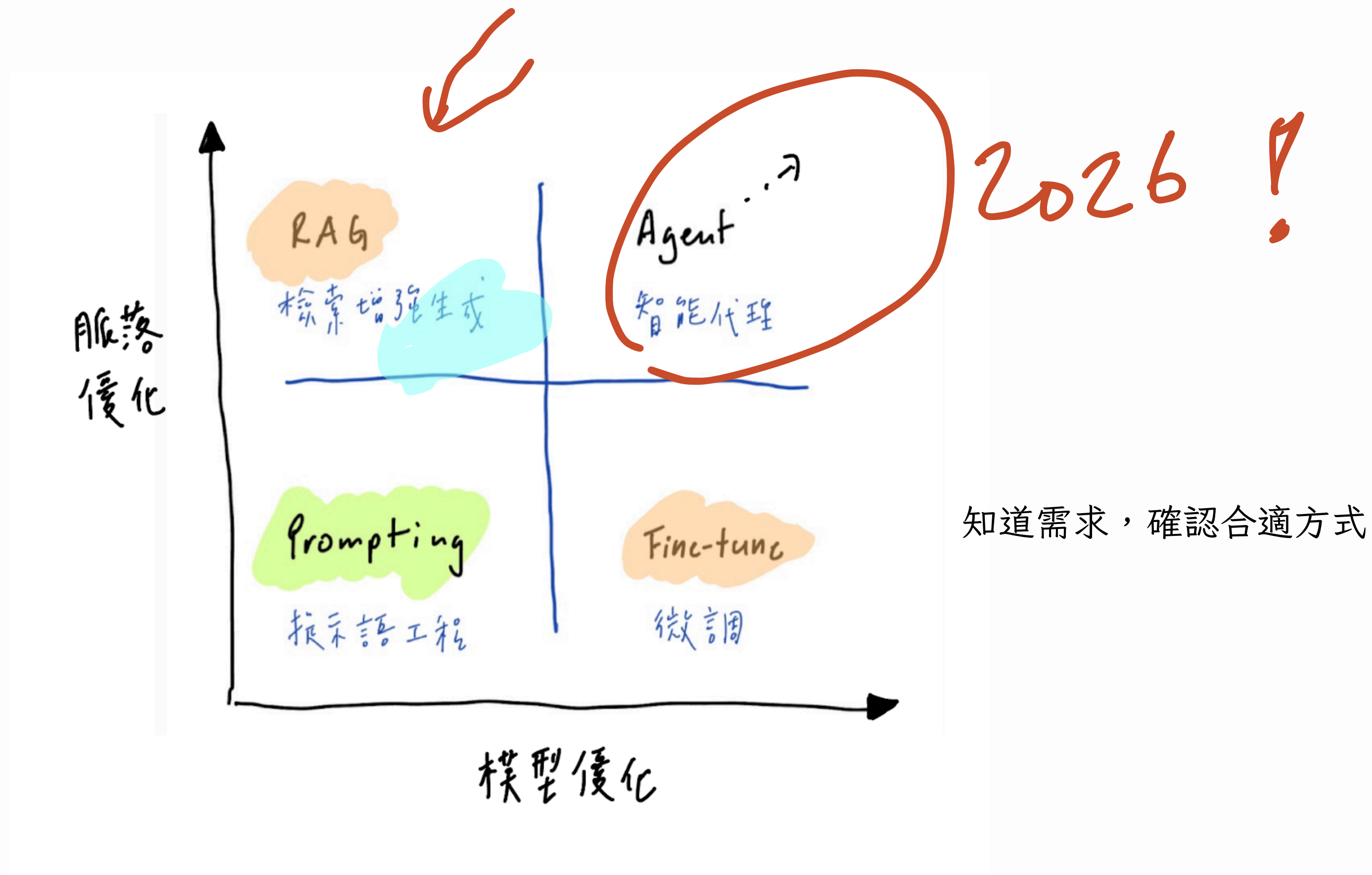
轉而產生應用！



<https://popbee.com/lifestyle/your-ai-slop-bores-me-human-ai-drawing-trend>

大型語言模型發展

2025 的教法



上週提到的幾個基本概念，在接下來幾週逐步深化

2

Agents 管理

多 Agent 系統：分工、溝通與階層

Multi-agent orchestration

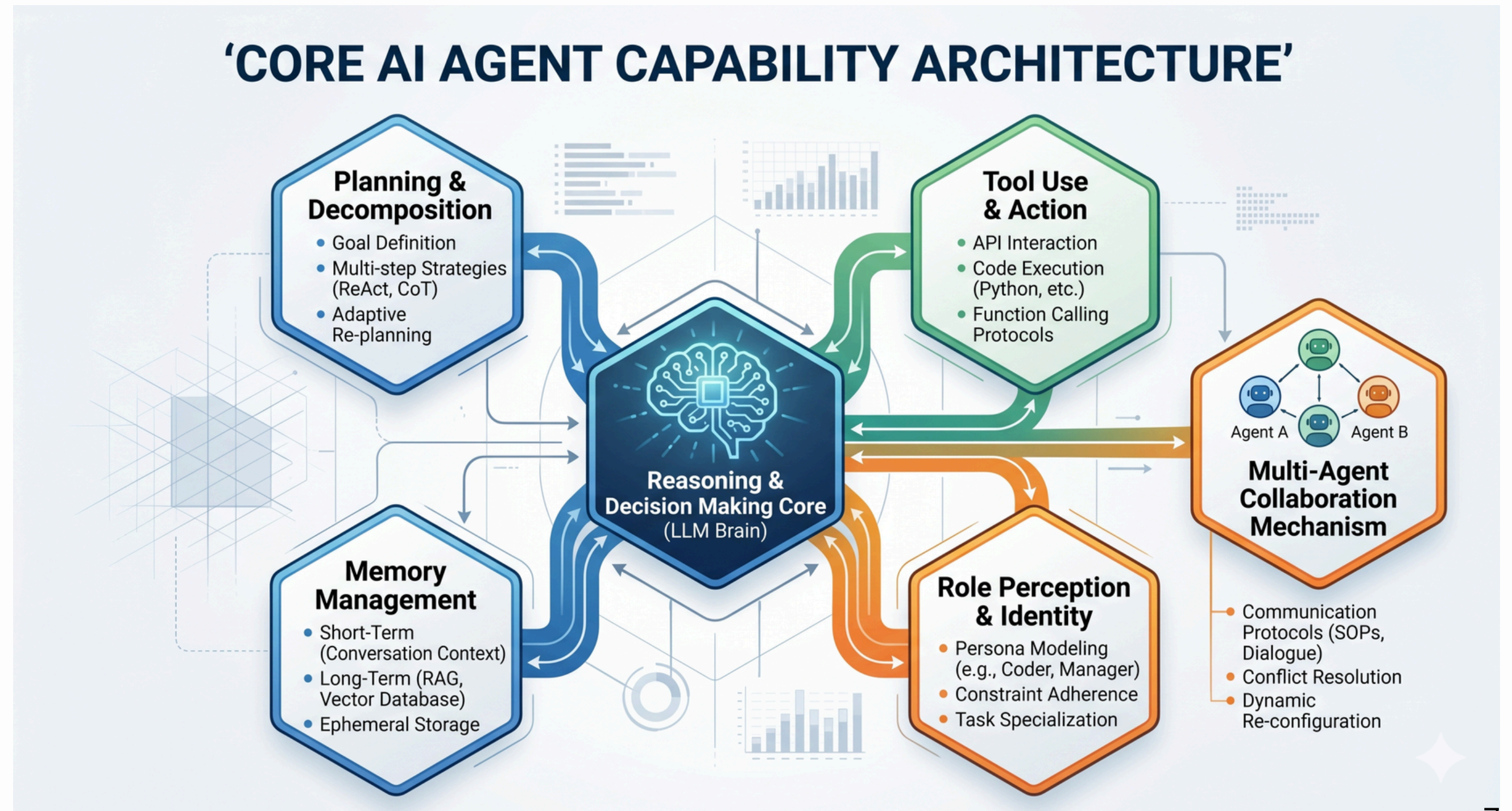
3

Agent 作為角色：扮演、視角模擬與思辨對話

4

人機互動：從來回到即時互動

Agent 的核心能力



模型優化到代理優化

模型優化到代理優化

優化模型我們之前略聊過，我們來談代理優化

Ask



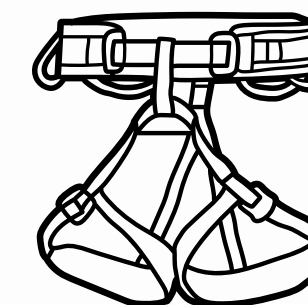
Prompt engineering

Know



Context engineering

Operate



Harness engineering

【如何跟一個 AI 說話】的演化史
概念許多重疊，但側重不同

三個工程範式的演化

從「怎麼說那一句話」到「怎麼給 AI 一個完整的工作環境」

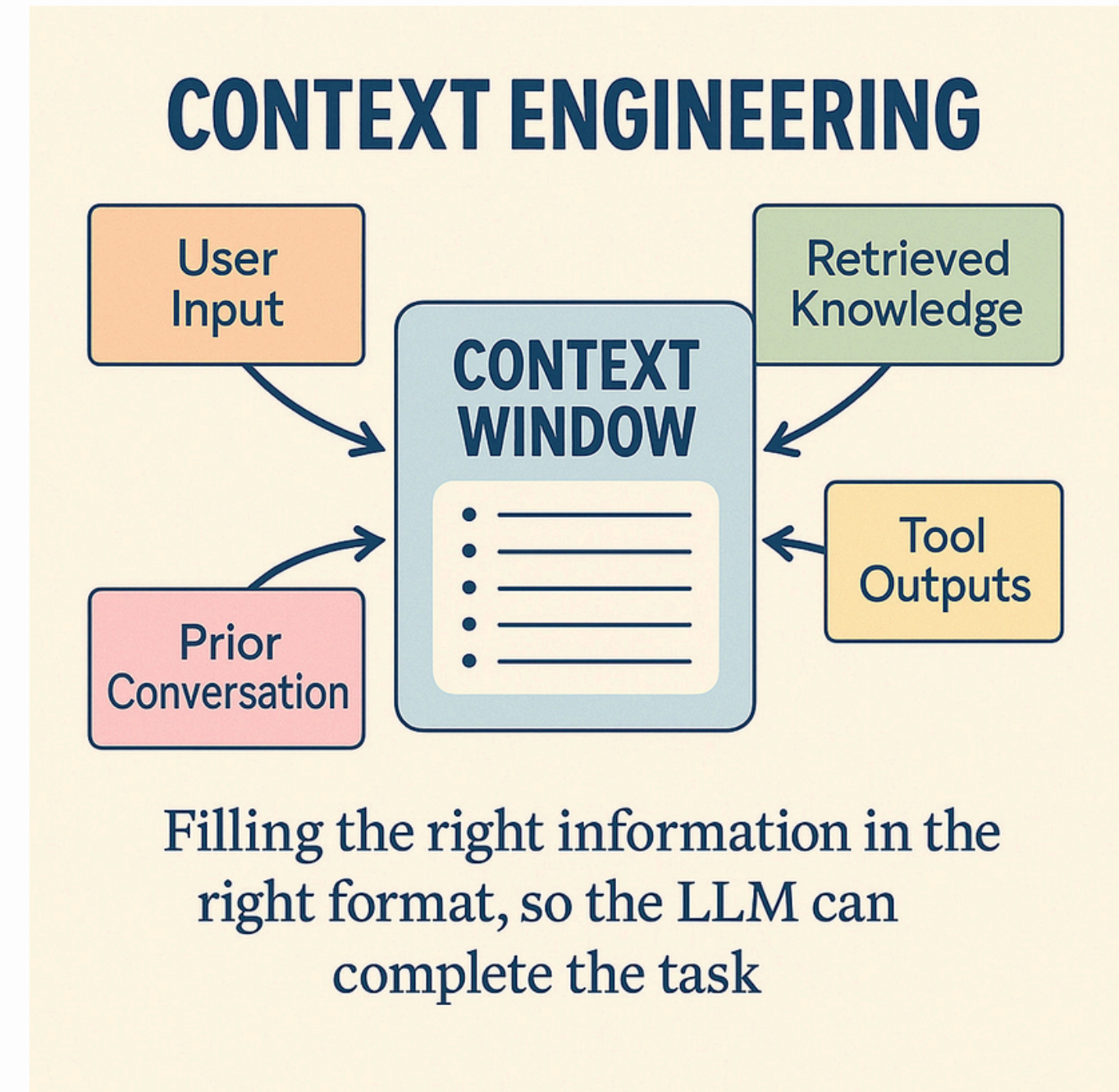


脈絡優化

Context Engineering

模型在生成下一個字時，當前視窗（Context Window）內所能看到的全部資訊。

形式一點的定義：模型在一次推理（Inference）中能處理的總 Token 數（包含系統提示詞、過去幾輪對話、檢索到的資料）。



Context Engineering：管理模型「看到什麼」

Context window 是 AI 的工作桌面 — 但桌面有限，你放什麼決定它能做什麼



Context Engineering：管理模型「看到什麼」

Context window 是 AI 的工作桌面 — 但桌面有限，你放什麼決定它能做什麼



檢索增強生成

RAG and alike

RAG (*Retrieval-Augmented Generation*)

RAG 是一種結合「檢索 (Retrieval)」與「生成 (Generation)」的 AI 發展模型架構。

核心理念是：「先查資料，再作答」，而不是像傳統大型語言模型 (LLM) 只憑「記憶」生成回答。簡言之，先搜尋出相關內容，放進 Prompt 裡面當作 context，再問 LLM。可有效減少 LLM 的幻覺現象，讓 LLM 根據資料進行回答。

為什麼需要 RAG？

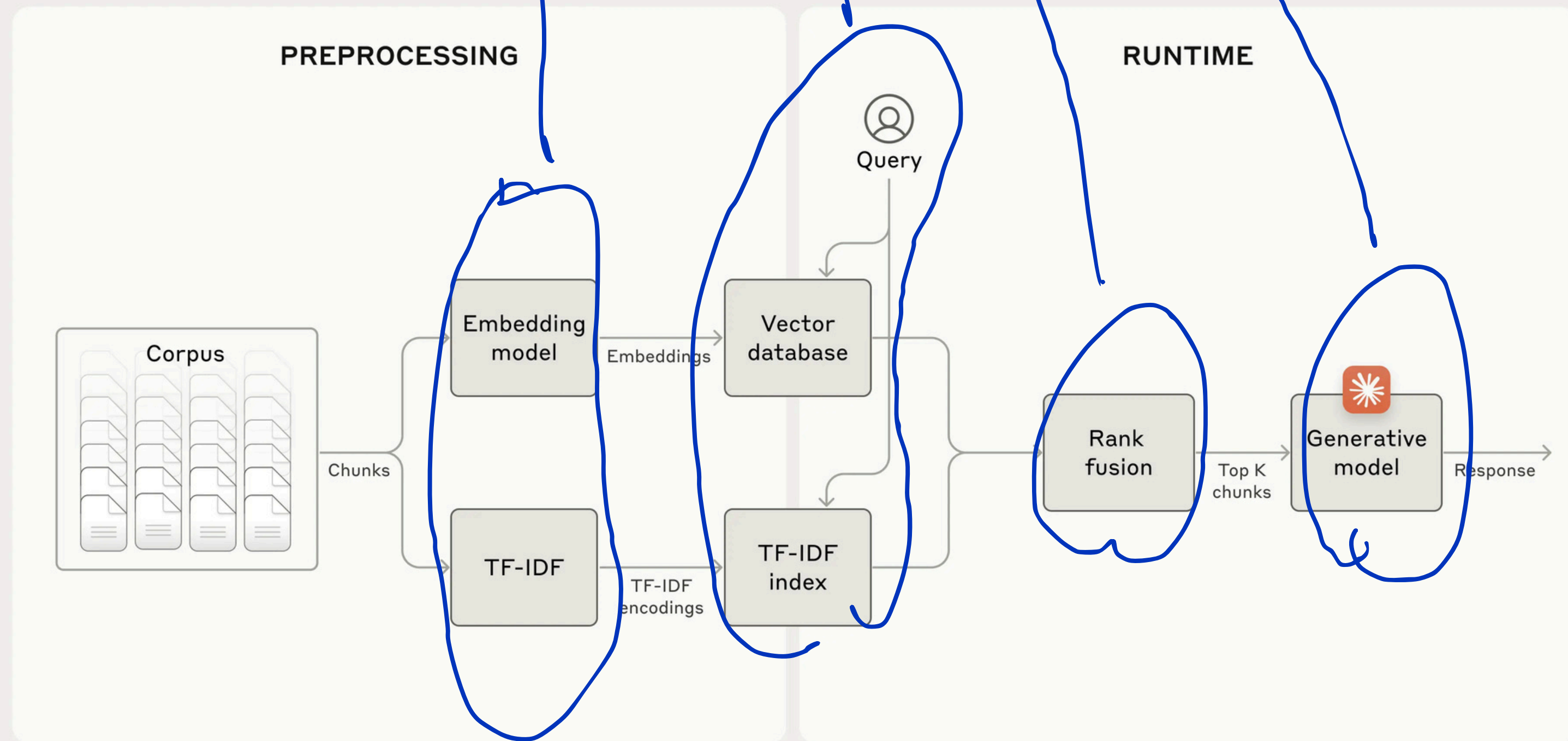
問題	RAG 解法
LLM 記憶有侷限	可接上外部知識庫，不受訓練資料限制
回答可能幻覺 (hallucination)	有檢索支撐，更能「有憑有據」地回答
實時知識更新困難	可連結即時資料 (如新聞、文件、企業內部資料)

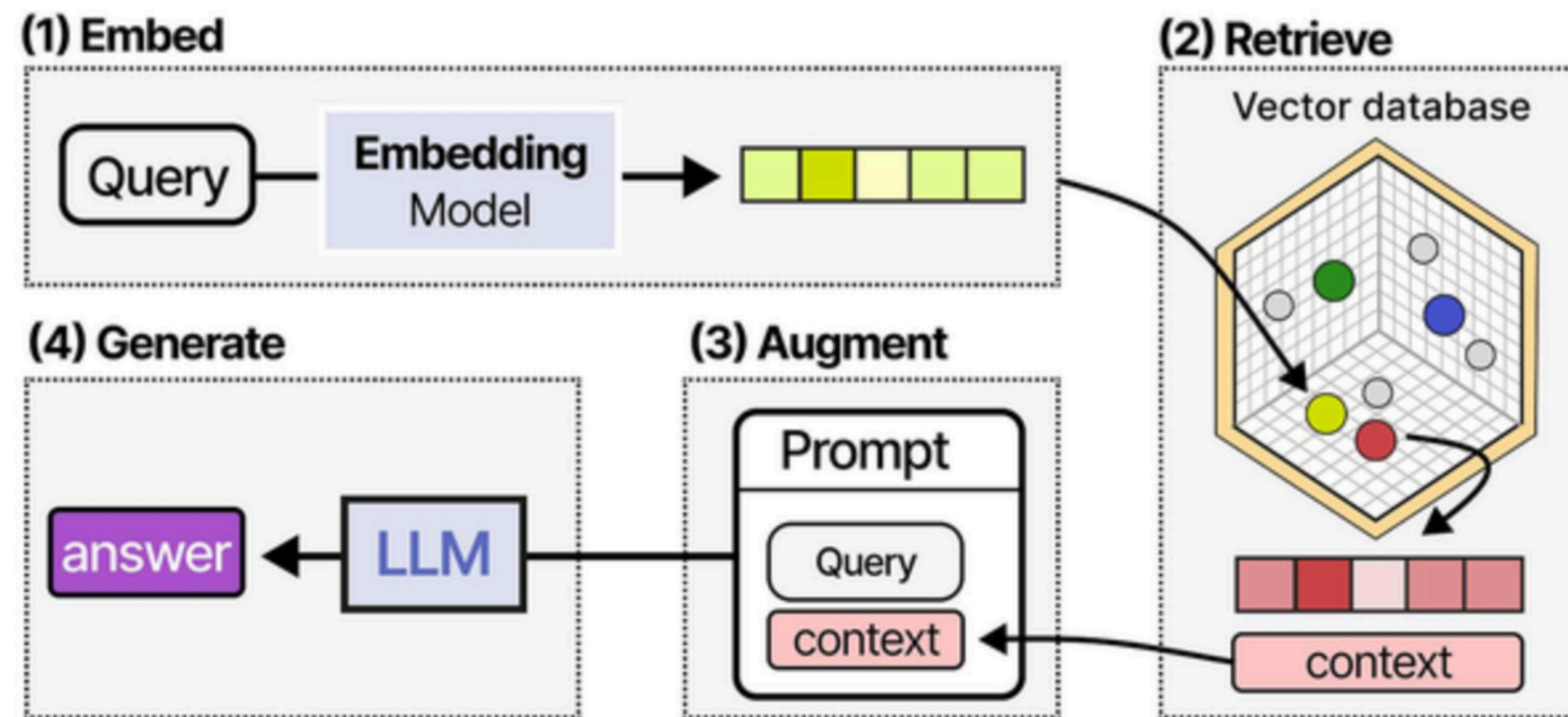
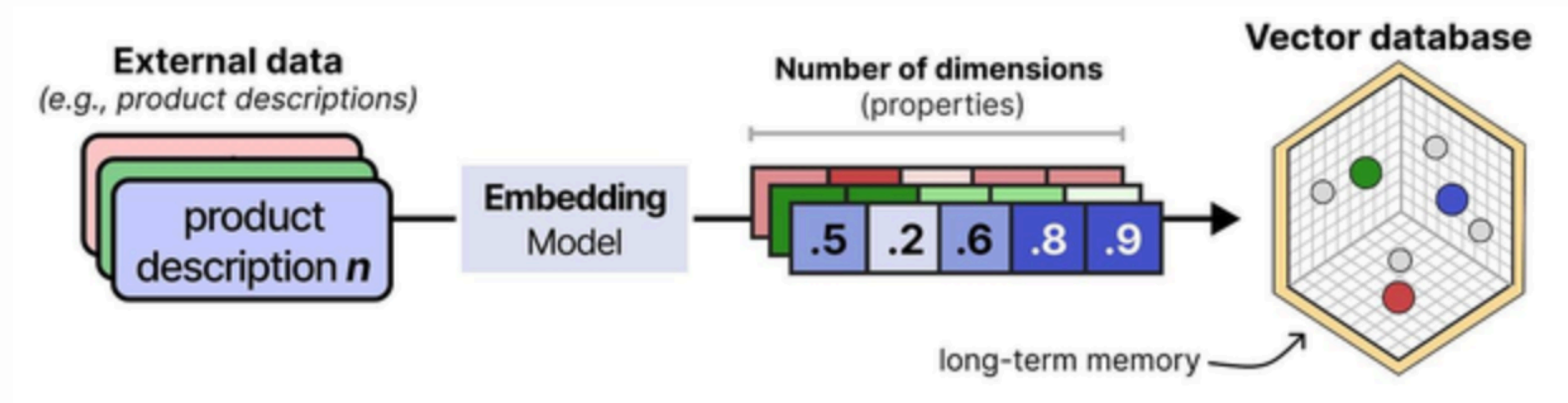
一般流程如下：

1. 使用者輸入問題（如：「海德格如何定義真理？」）
2. 檢索器從知識庫中找到與「真理」相關的段落（可能是《存在與時間》的某段）
3. 把這段內容（當成 **context**）和原問題一同送入 LLM
4. LLM 參考這些段落，生成回答

索引、檢索、排序、生成

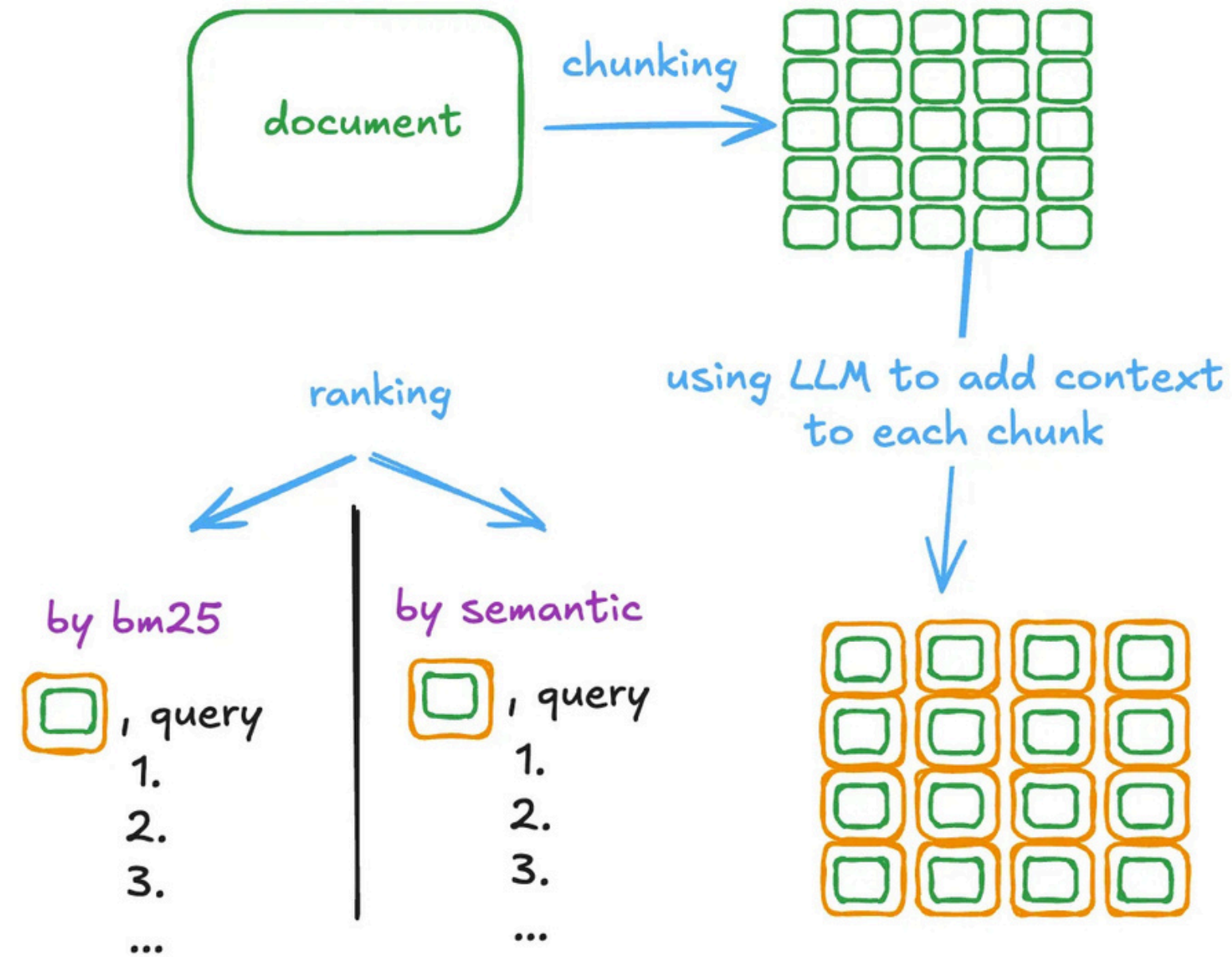
Standard RAG





Contextual Retrieval

contextual embeddings + contextual BM25



$$\text{final score} = \text{weight}_{\text{bm25}} * \left(\frac{1}{\text{rank}} \right)_{\text{bm25}} + \text{weight}_{\text{emb}} * \left(\frac{1}{\text{rank}} \right)_{\text{emb}}$$

人文應用案例

領域	應用說明
哲學	RAG 查找原典語錄、注釋與學者論述，協助生成「有出處」的哲學回答
歷史	透過檢索地方志、史書片段，進行年代或事件查詢
語言學	檢索語料庫中語法用例，幫助生成語法分析或例句
法律	查找判例與法條，生成具引用依據的法律建議 (Legal-RAG)

Context Engineering vs RAG

一個是「如何管理 context」，一個是「如何取得知識」

維度	CONTEXT ENGINEERING	RAG
解決的問題	context window 裡放什麼、怎麼管理	模型不知道某件事，怎麼讓它知道
操作對象	system prompt、history、documents、壓縮策略	向量資料庫、嵌入模型、相似度搜尋
需要外部資料庫？	不一定 可以只管理對話歷史	是 核心就是外部知識庫
即時性	管理已有的內容；不主動「取回」	每次對話前主動搜尋最新相關片段
包含關係	RAG 是 context engineering 的一個子技術 —— RAG 的輸出（取回的文件）最終要被注入 context，如何注入、注入多少，就是 context engineering 的工作。	
典型工具	LangChain Memory、自訂 history 管理、prompt caching	nomic-embed-text、Chroma、FAISS、LlamaIndex
人社研究應用	控制 AI 扮演的角色、管理長篇田野筆記分析的歷史	把一千篇文獻存成向量庫，問題來了就取最相關的幾篇

Context Engineering vs RAG

一個是「如何管理 context」，一個是「如何取得知識」

維度	CONTEXT ENGINEERING	RAG
解決的問題	context window 裡放什麼、怎麼管理	模型不知道某件事，怎麼讓它知道
操作對象	system prompt、history、documents、壓縮策略	向量資料庫、嵌入模型、相似度搜尋

最精確的定位

Context engineering 是「戰略」，RAG 是「戰術」。你先決定資訊管理策略（CE），再決定知識怎麼取回（RAG）。

什麼時候只用 CE？

任務不需要外部知識——只靠模型訓練知識加上對話歷史就夠時。例如：寫作助理、邏輯推理任務。

什麼時候 CE + RAG 並用？

需要最新知識、大量文件查詢、或模型訓練資料不涵蓋的領域。例如：企業內部知識庫問答、最新論文分析。

Context Engineering vs RAG

一個是「如何管理 context」，一個是「如何取得知識」

維度	CONTEXT ENGINEERING	RAG
解決的問題	context window 裡放什麼、怎麼管理	模型不知道某件事，怎麼讓它知道
操作對象	system prompt、history、documents、壓縮策略	向量資料庫、嵌入模型、相似度搜尋

Context Engineering

管理整個工作桌面
決定 context window 裡放什麼、放多少、如何壓縮——是一個全局性的設計，涵蓋 system prompt、history、documents、query 的組合策略。

範疇：整個 context window 的構成與生命週期管理

共同點

都在管理 AI 看到的資訊

都影響回答品質

都需要 token 預算意識

RAG

從外部取得知識
Retrieval-Augmented Generation——把問題轉成向量，從外部資料庫取回相關段落，再注入 context 的特定技術流程。

範疇：external memory → in-context 的取回機制

最精確的定位

Context engineering 是「戰略」，RAG 是「戰術」。你先決定資訊管理策略（CE），再決定知識怎麼取回（RAG）。

什麼時候只用 CE？

任務不需要外部知識——只靠模型訓練知識加上對話歷史就夠時。例如：寫作助理、邏輯推理任務。

什麼時候 CE + RAG 並用？

需要最新知識、大量文件查詢、或模型訓練資料不涵蓋的領域。例如：企業內部知識庫問答、最新論文分析。

Context Engineering：Context Compression

當 context window 快滿了，你必須做決定：脈絡壓縮

策略 01
滑動視窗
Sliding Window

只保留最近 N 輪對話，舊的直接丟棄。實作最簡單，但早期的重要脈絡會永遠消失。

資訊損失：高

策略 02
摘要壓縮
Summarization

用另一個 LLM 呼叫把舊對話濃縮成摘要，再放回 context。保留語義但失去細節，且摘要本身也可能有幻覺。

資訊損失：中

策略 03
選擇性保留
Selective Retention

根據當前問題的相關性，動態決定哪些歷史片段值得保留。需要語義判斷，成本較高但效果最好。

資訊損失：低

策略 04
結構化萃取
Structured Extraction

把對話中的關鍵事實抽取成結構化格式（JSON / 列表）儲存，後續只注入結構，不注入原始對話。

資訊損失：低（事實層）

● 壓縮發生的時機

- 主動壓縮 每 N 輪自動摘要，或 token 用量超過閾值時觸發
- 被動截斷 超過 window 上限時，API 自動截斷最舊的訊息
- 任務結束 對話結束後，萃取關鍵資訊存入 external memory

● 壓縮的哲學問題

- 選擇偏差 「重要」是誰定義的？摘要本身就是一種詮釋
- 細節 vs 脈絡 壓縮保留語義但失去文字肌理——影響創意任務
- 幻覺風險 摘要 AI 可能扭曲原意，再次注入後誤導推理

Context Engineering：Context Compression

脈絡壓縮就是一種主動的記憶管理

- Context compression 不只是「省 token 的技巧」。它本質上是一種記憶轉移操作：把 in-context memory 轉存為 external memory，等需要時再取回。這個循環的設計，決定了 AI 在長任務中「記得什麼、忘了什麼」。



記憶

Memory

Memory 的四種類型

類型 01

In-context memory

工作記憶（短期）

儲存位置：context window

對話過程中模型「看到」的所有文字——你說的話、它的回答、注入的文件。對話結束就消失。

例：「你剛才說的那個論點……」→ AI 能回應，因為它就在 context 裡。

上限：幾萬到幾十萬 token（依模型而定）

類型 02

External memory

外部記憶（長期）

儲存位置：資料庫 / 向量庫

把重要資訊存在外部（文字檔、向量資料庫），需要時用 RAG 檢索再注入 context。跨對話持久存在。

例：把一千篇論文的摘要存成向量，AI 每次只取最相關的幾篇來回答。

上限：幾乎無限，但每次只能注入一小部分

類型 03

In-weights memory

參數記憶（訓練知識）

儲存位置：模型權重

模型在訓練時學到的所有知識，壓縮在數十億個參數裡。你無法在使用時修改它。

例：AI「知道」魯迅是誰，不需要你告訴它——這是訓練時學的。

固定：訓練結束後不再更新（除非 fine-tuning）

類型 04

In-cache memory

快取記憶（KV cache）

儲存位置：GPU 記憶體

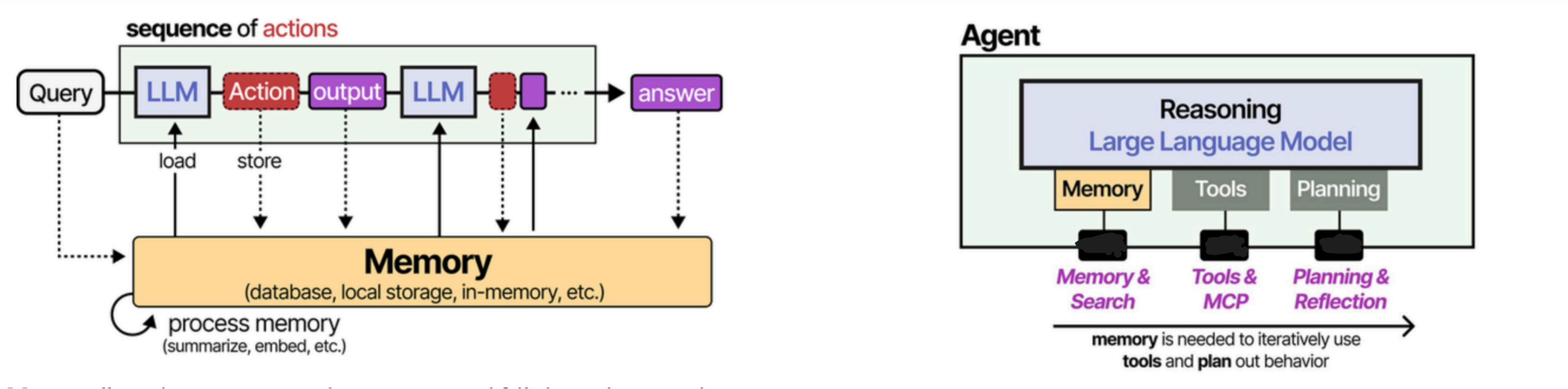
AI 計算過的 attention 結果暫存在硬體快取，讓重複出現的 context 不需要重新計算，加快推理速度。

例：Anthropic 的 prompt caching——同樣的 system prompt 只計算一次，降低費用。

工程最佳化層，使用者通常不需要直接管理

Context engineering 的核心工作：決定什麼放進 in-context，怎麼從 external memory 取出最有用的部分——在有限的 token 預算裡做出最好的選擇。

Agent Memory 顯得更為重要



程式示範：Memory 在 Ollama 中的運作

同樣的問題，有無帶入對話歷史，結果截然不同

無記憶版

```
# 每次呼叫都是全新的對話
def ask(prompt):
    r = requests.post(
        "http://localhost:11434/api/chat",
        json={"model": "gemma3:4b",
              "messages": [
                  {"role": "user", "content": prompt}
              ], "stream": False})
    return r.json()["message"]["content"]

> ask("魯迅的本名是什麼？")
"魯迅本名周樹人..."

> ask("他出生在哪裡？")
"請問您指的是誰？"
# ← 失憶了，不知道「他」是誰
```

對話結束後，模型對「他」一無所知

有記憶版 (帶 history)

```
# 把對話歷史帶進每次呼叫
history = []

def chat(user_msg):
    history.append(
        {"role": "user", "content": user_msg})
    r = requests.post(
        "http://localhost:11434/api/chat",
        json={"model": "gemma3:4b",
              "messages": history, # ← 關鍵
              "stream": False})
    reply = r.json()["message"]["content"]
    history.append(
        {"role": "assistant", "content": reply})
    return reply

> chat("魯迅的本名是什麼？")
"魯迅本名周樹人..."
> chat("他出生在哪裡？")
"周樹人出生於浙江紹興..." # ← 記住了
```

「記憶」不是 AI 的能力，是你每次呼叫帶入歷史的工程決策

RAG (Retrieval-Augmented Generation) 概念流程

Step 01
使用者問題
轉成向量

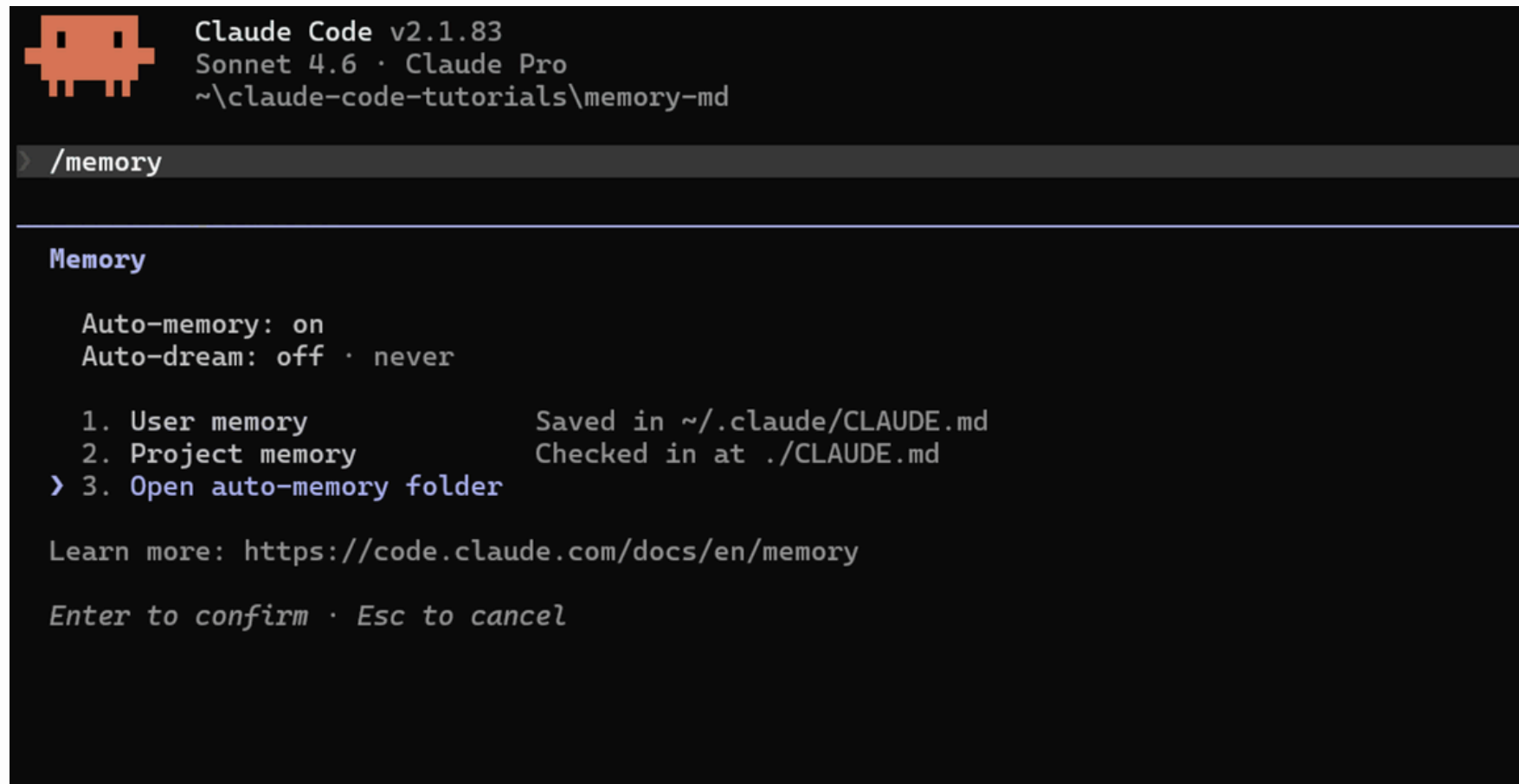
Step 02
向量搜尋
找出相關段落

Step 03
注入 context
段落 + 問題

Step 04
模型回答
有根據的回應

Claude code 的記憶功能

- 會在你 coding 時，自動把專案背景、除錯心得、偏好設定、常用指令等記錄下來，存成 MEMORY.md 檔案。
- Auto Memory 讓agent邊工作邊記筆記；AutoDream 則是你睡一覺後，agent 把筆記整理、合併、刪掉廢紙、寫成乾淨摘要



```
Claude Code v2.1.83
Sonnet 4.6 · Claude Pro
~\claude-code-tutorials\memory-md

> /memory

Memory

Auto-memory: on
Auto-dream: off · never

1. User memory          Saved in ~/.claude/CLAUDE.md
2. Project memory       Checked in at ./CLAUDE.md
> 3. Open auto-memory folder

Learn more: https://code.claude.com/docs/en/memory

Enter to confirm · Esc to cancel
```

工具使用

Tool use

Tool use

Tool use 的運作是一個感知→決策→行動→觀察的循環，這個循環可以重複多次，形成上次提到所謂的 ReAct 模式（Reasoning + Acting）

- 三種常見【工具】類型

類型	例子	對應能力
資訊檢索	網路搜尋、資料庫查詢、RAG	眼睛（感知）
計算與執行	執行 Python、呼叫 API	手（行動）
環境互動	讀寫檔案、操作瀏覽器、寄信	身體（embodiment）

- Tool use 在語用學上很有趣：模型必須解讀用戶意圖（illocutionary force），判斷「這句話需要我去查資料，還是直接回答就夠？」

Tool use

直接問模型不好嗎？

請算出這句話的字數與詞數：「語言是存在的家」

💡 Thought for 14.0 seconds

這句話「語言是存在的家」的字數和詞數如下：

- **字數 (Characters Count): 7 個**
 - (語、言、是、存、在、的、家)
- **詞數 (Word Count):** 依不同的語法分析方式而定，但如果以標準的語義詞塊（如「語言」、「存在」等）來計，可以區分為幾個語法單元，但最精確且通用的計數是7個字。

總結：這句話有 7 個字。



詞數與字數不會一樣，請再算一次

💡 Thought for 15.1 seconds

您說得非常正確，字數和詞數在中文語法分析中是兩個不同的概念。我前一次的回答過於字面化，沒有按照語義單位來計算。

tool_test.py > ...

```
1  import ollama
2  import json
3  import jieba
4
5  # 定義工具（告訴模型「你有什麼手可用」）
6  tools = [
7      {
8          "type": "function",
9          "function": {
10             "name": "get_word_count",
11             "description": "計算一段文字的字數與詞數",
12             "parameters": {
13                 "type": "object",
14                 "properties": {
15                     "text": {
16                         "type": "string",
17                         "description": "要計算的文字"
18                     }
19                 },
20                 "required": ["text"]
21             }
22         }
23     ]
24
25
```

● (base) → week8 python tool_test.py

Building prefix dict from the default dictionary ...

Loading model from cache /var/folders/r7/zvnd0d1x0mzb04qcszmsss2m0000gn/T/jieba.cache

Loading model cost 0.299 seconds.

Prefix dict has been built successfully.

[工具呼叫] get_word_count({'text': '語言是存在的家'}) → {'characters': 7, 'words': 5}

[最終回覆] 這句話「語言是存在的家」包含：

* **字數（字）：** 7 個字

* **詞數（詞）：** 5 個詞

【分析】

在中文的詞語概念中，通常會將「語言」、「是」、「存在」、「的」、「家」視為單獨的詞語或語組。因此，按詞彙單位計算為 5 個詞。字數則是指文本中所有的字符數量，為 7 個字。

電腦使用

Computer use

Claude Computer Use

讓 AI 直接操作你的電腦，搭配 Dispatch 手機遠端指揮

- 點滑鼠、打鍵盤、開 app、填試算表、瀏覽網頁。



Claude Computer Use 教學：讓 AI 直接操作你的電腦，搭配 Dispatch 手機遠端指揮

Anthropic 推出 Claude Computer Use，讓 Claude 在 Cowork 和 Claude Code 中直接操作你的 Mac 電腦——點擊、打字、瀏覽網頁、填表格。搭配 Dispatch 功能，還能用手機遠端指派任務。本文介紹運作原理、三層工具優先級、安全機制、設定步驟，以及設...

Computer use

(Claude) 請打開我的瀏覽器，到教育部萌典網站 (<https://www.moedict.tw>) 找臺灣台語辭典中的諺語部分，並交給諺語中有數字的部分區塊起來。

請確認以下幾點：

- Chrome 已開啟，且已安裝「Claude in Chrome」擴充功能（extension）。
- Extension 已啟用：在 Chrome 右上角應可看到 Claude 的圖示。
- 已點擊「Connect」：第一次使用時，擴充功能需要在瀏覽器端點擊連接按鈕，才能讓 Claude 控制瀏覽器。

Agents 管理 多 Agent 系統：分工、溝通與階層

Multi-agent orchestration

MAS 定義與動機

A multi-agent system consists of multiple autonomous agents that interact, coordinate, and collaborate toward a shared or competing objective.

為什麼一個很強的 LLM 還需要多個 agent ？

因為 multi-agent 的核心不是模型更大，而是分工與協調。

One model is intelligence; multiple agents are organization.

Role specialization

Agent	Function
Planner	任務拆解
Researcher	搜尋資訊
Executor	寫 code / 執行工具
Critic	檢查錯誤
Memory Agent	維持上下文

Multi-Agent 三種協作架構

Architectures determines how reasoning flows.



注意不是不同模型才叫 multi-agent，同一個 local model 搭配不同 system prompt (角色設定) 也可以形成 agent team。此外，MAS 中，另一個 agent 本身也可以是工具。

合作機制

Coordination Mechanisms

- agents 不是「輪流講話」而已。
 - 成功的合作，包括了訊息傳遞 message passing、共享記憶 shared memory、呼叫工具 tool calling、共享黑板 blackboard architecture、共識形成 voting / debate 等。
 - 失敗的合作，包括了幻覺傳遞 hallucinated coordination、角色崩壞 role collapse、無窮迴圈 infinite loops、矛盾 contradiction、重複工作 redundant work 等。

評測基準

Benchmarks

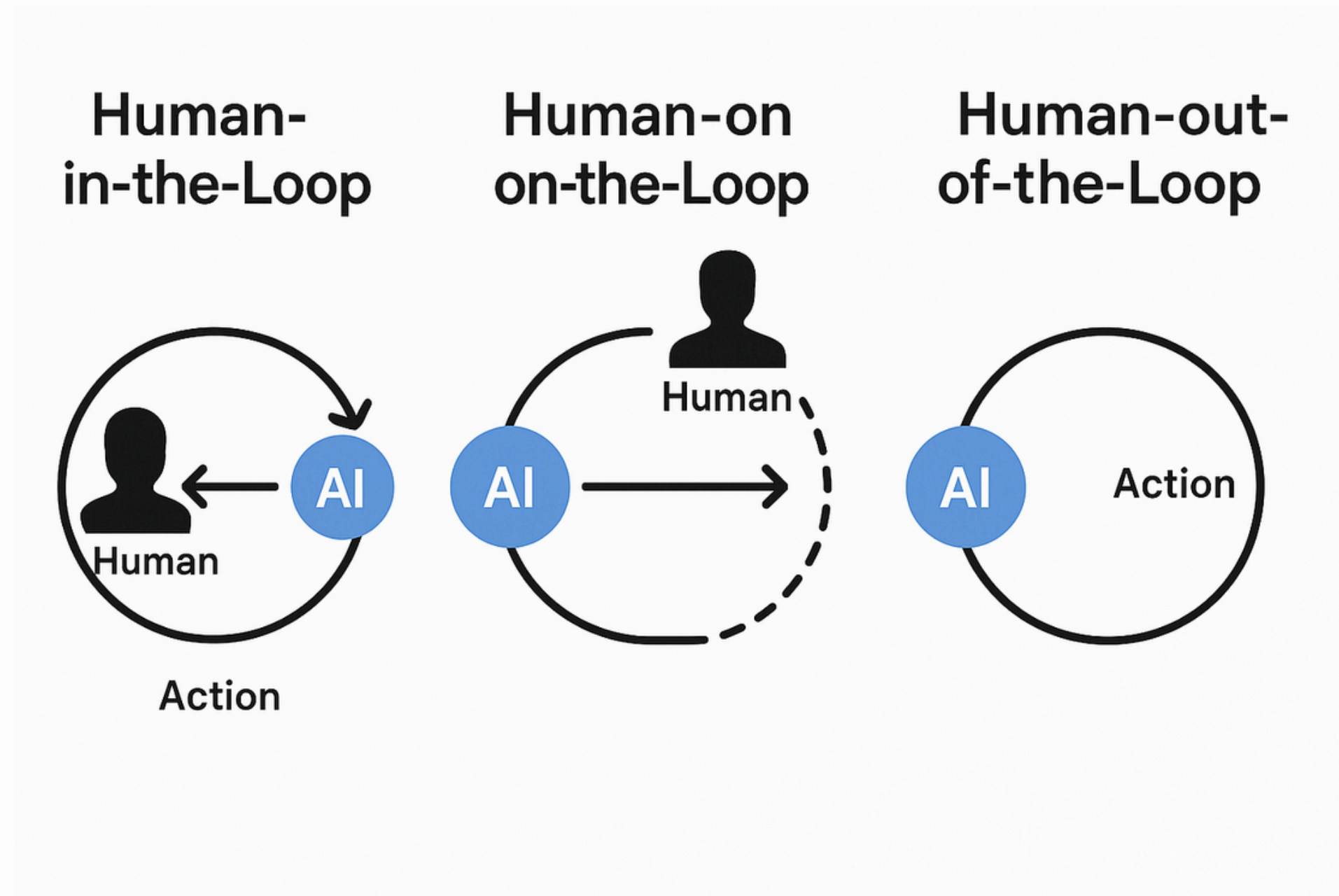
Metric	Meaning
accuracy	任務完成度
latency	時間
token cost	成本
consistency	agent 間一致性
explainability	推理透明度

五個控制問題

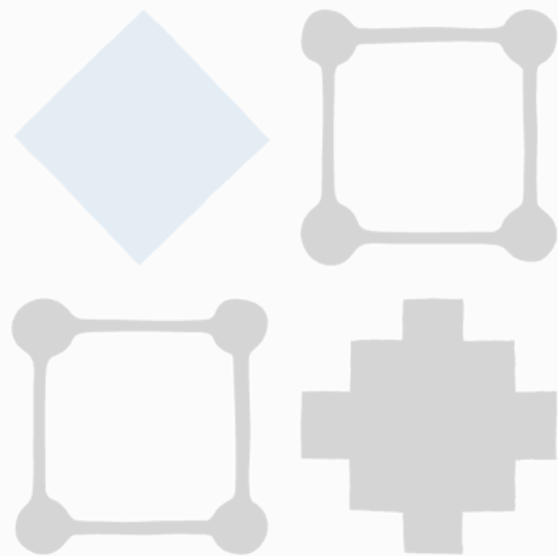
設計 Multi-Agent 系統前，這些問題沒有回答，系統就不該上線

設計問題	業界常見做法
01 誰是協調者？ 任務要分給哪個 Agent？由誰決定分派？	設置 Orchestrator Agent 負責分解任務，或用 LangGraph 的圖結構預先定義流程。 CrewAI manager / LangGraph supervisor node
02 Agent 之間怎麼溝通？ 傳什麼格式？同步還是非同步？	結構化訊息傳遞（JSON schema）；或共享狀態物件，所有 Agent 都能讀寫。 LangGraph State dict / CrewAI task output
03 記憶怎麼共享？ 每個 Agent 都有自己的記憶，還是有公共記憶庫？	分兩層：私有 context（每個 Agent 的任務歷史）+ 共享 external memory（向量資料庫，所有 Agent 都可查）。 shared vector store + private conversation history
04 出錯怎麼辦？ Agent 失敗、產生幻覺、陷入無限迴圈……	設定最大迭代次數、加入 Validator Agent 檢查輸出、設計 fallback 路徑。 未設計錯誤處理 = 生產環境最大風險
05 人在哪裡介入？ 全自動還是要人審核？誰負責？	Human-in-the-loop 不是選項，而是設計原則——高風險決策點（刪除資料、送出訊息）必須要求人類確認。 LangGraph interrupt() / CrewAI human_input=True

那人的角色呢？



Engineering at Anthropic



Harness design for long-running
application development

駕馭工程

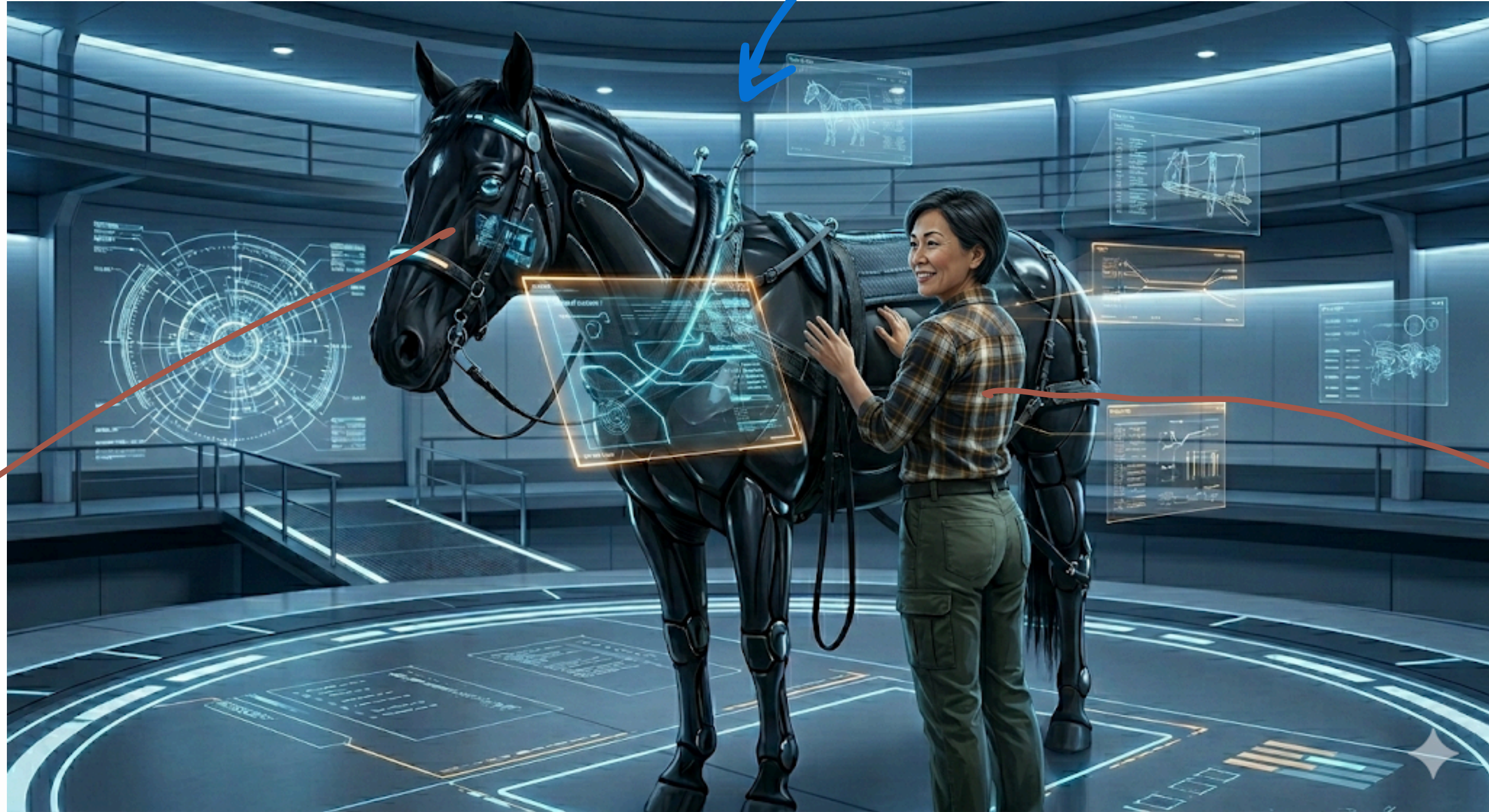
Harness Engineering

Published Mar 24, 2026

Harness design is key to performance at the frontier of agentic coding. Here's how

模型夠強還沒有，需要一個好的工作環境讓其發揮

Harness:一整套控制馬匹的工具（韁繩、馬鞍、馬具等）



AI

AI
工程師

有哪些「鞍具」(來控制模型的產出)？

- 語言：寫出的規則、規格

Agents.md CLAUDE.md

- 工具

- 流程

不是教模型怎麼回答，而是設計模型如何工作、系統如何運轉（任務交接、上下文管理、工具編排、權限設定、狀態交接、失敗重試、驗證、恢復讓人接管等）

因為幾句話 prompting 搞不定

人文省思

Human-in-the-loop 是技術選項，還是倫理義務？當 AI Agent 代表你做決定，「我不知道它做了什麼」
是可以接受的答案嗎？

前五個問題不只是工程問題，也是 alignment 問題的縮影：誰有控制權、誰負責任、如何驗證 AI 按照你的意圖行事？

實作課

實務上，如何使用 AI

擅用專業版功能：以 Claude 為例

- 設定檔 | profile
- 開啓記憶功能 | settings - capabilities ； (按其指示) 匯入chatGPT 記憶
- 接上 connectord | settings - capabilities ； (按其指示) 匯入chatGPT 記憶
- skills
- （在對話視窗講也可）安裝擴充功能 Claude in Chrome
- 設定 computer use ：安裝 Claude 桌面應用程式（Claude Desktop），在 Cowork 或 Claude Code 中啓用，而非在 claude.ai 網頁版中操作。）

期末展演的規劃（小更正）

鼓勵小組合作，理想上 3-4 人，
歡迎跨校合作。打算個人實作需
要先行跟老師助教提出申請。

- 第九週 4/24

詢問非台大的鏡像學校學生是否參加實體展演（非強制）

- 第十一週 5/8

確認「非台大的鏡像學校學生」是否到台大參加實體展演；若不參加實體，則請在期末繳交自製錄影影片講解專題。

- 第十六週 6/12

期末展演（地點另行通知）。參加者全體台大學生 + 選擇實體報告的非台大學生，中南部學生可以酌情補助交通費。

海報展演時間（10:00-14:00）。由老師與助教群先進行評分，之後校外評審到場，進行最後決選、頒獎。

不厭其煩再提醒

人類在 Coding 工作上的重點，放在

- 定義問題
- 設定規則
- 提供創造性直覺
- 設計 AI 們的工作環境



都跟自然語言有關！

基本素養：語言概念分析表述與技術101

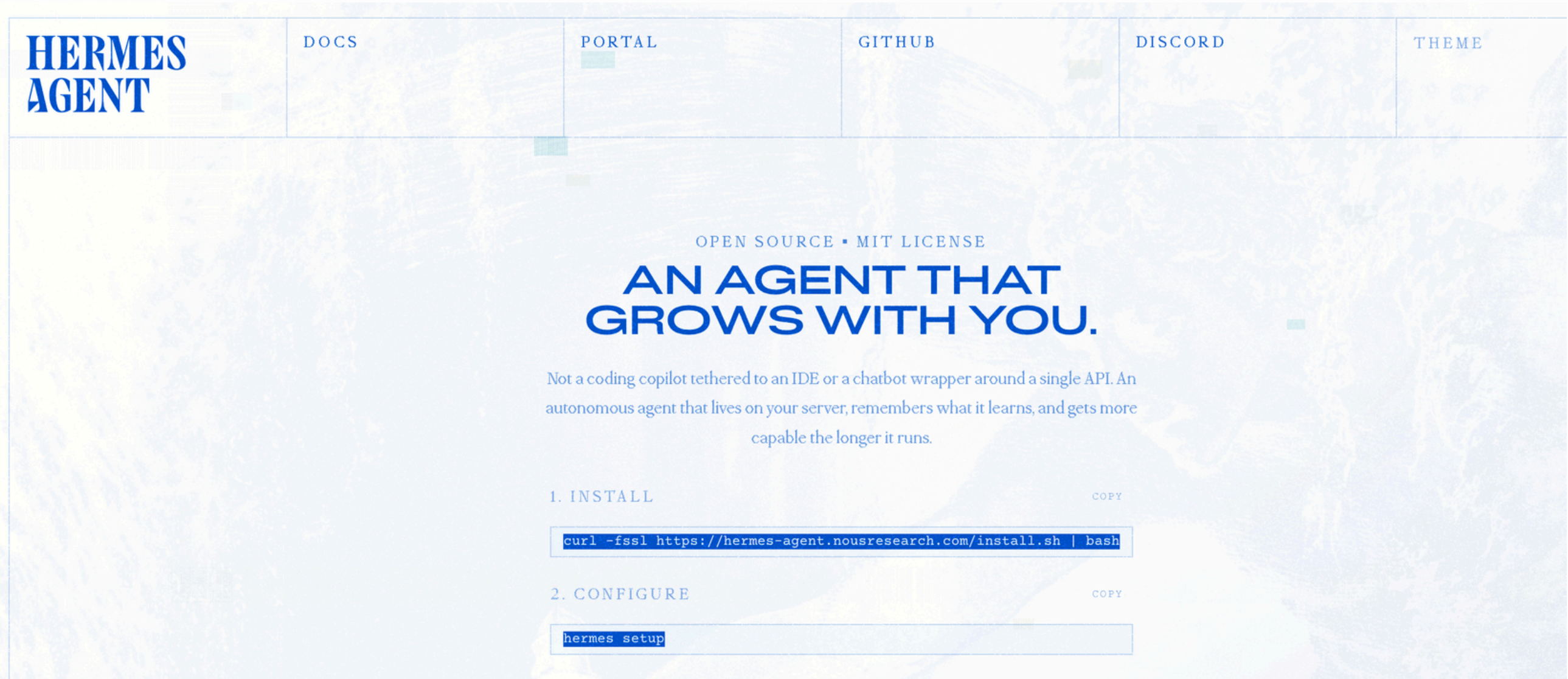
複習基本指令列 CLI

如何在地端運行 LLM (via Ollama)

Markdown 語法入門

預告：下次帶大家安裝 Hermes Agent

- 先在手機下載 Telegram 並申請帳號
- 確定 Ollama 可以運行



本週：指令列、本地 AI 環境、Markdown

◆ 生成式 AI 的人文導論

首頁課程進度課堂作業期末專案學習資源lab sessions

Session 4 · Lab

Markdown 語法與 AGENTS.md 實作

← 返回 lab sessions

Markdown 入門與 AGENTS.md 產生器

Markdown 核心語法、即時預覽編輯器與 AGENTS.md 互動式產生器